that have any kind of case conversion functionality are simply wrappers around internal calls to `mb_convert_case()`, with particular modes specified. Here are a few examples:

```
$str = 'wHo cAreS aBoUt ThE fIfTh eArl oF bAtHsDrOp aNd lAdY hIgGeNboTtOm';

print mb_convert_case($str, MB_CASE_UPPER) . PHP_EOL;
print mb_convert_case($str, MB_CASE_LOWER) . PHP_EOL;
print mb_convert_case($str, MB_CASE_TITLE) . PHP_EOL;

Output:
WHO CARES ABOUT THE FIFTH EARL OF BATHSDROP AND LADY HIGGENBOTTOM
who cares about the fifth earl of bathsdrop and lady higgenbottom
Who Cares About The Fifth Earl Of Bathsdrop And Lady Higgenbottom
```

The code above calls `mb_convert_case()` with three different modes, specified with the constants `MB_CASE_UPPER`, `MB_CASE_LOWER`, and `MB_CASE_TITLE`. And here's how those calls work with ASCII data: exactly the same as `strtolower()`, `strtoupper()`, and `ucwords()` – and I really do mean *exactly* the same. `mb_convert_case()`, as a performance enhancement, will pass any ASCII bytes into the same fast byte value manipulation that `strtolower()` et al. uses. Otherwise, a mapping operation is performed, which takes the Unicode codepoint ordinal of the source character and applies some rules for changing it to a new ordinal value (and then swapping the old byte values for the ones that match the new character). But how are these mapping rules defined? Well, remember the difficulties with Turkish `I` and `İ` characters. I mentioned that this wasn't a bug with `mb_strtolower()`, and it isn't: the function simply implements one of the Unicode standards for case mapping, and the problem lies there.

Case mapping simply means "one-to-one character mappings for bicameral alphabets" (e.g., bicameral alphabets have two characters for the same letter – `A` and `a`, `B` and `b`). Naturally, this mapping is simple for the basic Latin alphabet. Things start to become more challenging when trying to incorporate characters from, well, all of the alphabets, and the Unicode standard has a section entirely devoted to the various complications arising from locale, context, and the language conventions of particular character sets. You can read about them in section 5.18 of https://www.unicode.org/versions/Unicode15.1.0/ch05.pdf

In order to solve the edge cases of messy human language, Unicode defines "normal" and "special" mappings. The "normal" rules provide a base of one-to-one character mappings, with "special" rules dealing with complications arising from locale, context,